

Title

Application of Substitution Method for Big Data Analytics Pipeline

Aim

To solve the recurrence relation

$$T(n) = T(n - 1) + \log n$$

using the substitution method and determine its asymptotic complexity.

Theory

The recurrence

$$T(n) = T(n - 1) + \log n$$

indicates that each processing stage performs all previous work plus an additional logarithmic amount of computation.

This type of recurrence commonly appears in:

- Big Data Analytics
- Incremental data processing
- Streaming algorithms
- Log-based indexing

Step-by-Step Expansion

Given

$$T(n) = T(n - 1) + \log n$$

Expand repeatedly:

$$T(n) = T(n - 2) + \log(n - 1) + \log n$$

Again,

$$T(n) = T(n - 3) + \log(n - 2) + \log(n - 1) + \log n$$

Continuing,

$$T(n) = T(1) + \sum_{i=2}^n \log i$$

Since

$$T(1) = 1$$

we get

$$T(n) = 1 + \sum_{i=2}^n \log i$$

Using the logarithm property,

$$\begin{aligned} \sum_{i=2}^n \log i &= \log(2 \times 3 \times 4 \times \cdots \times n) \\ &= \log(n!) \end{aligned}$$

Therefore,

$$T(n) = 1 + \log(n!)$$

Using Stirling's approximation,

$$\log(n!) = \Theta(n \log n)$$

Hence,

$$\boxed{T(n) = \Theta(n \log n)}$$

Complexity Derivation

Using substitution,

$$T(n) = 1 + \log(n!)$$

Since

$$\log(n!) = \Theta(n \log n)$$

Therefore,

$$\boxed{T(n) = \Theta(n \log n)}$$

Time Complexity

Case	Complexity
------	------------

Best Case	$\Theta(n \log n)$
-----------	--------------------

Average Case	$\Theta(n \log n)$
--------------	--------------------

Worst Case	$\Theta(n \log n)$
------------	--------------------

Space Complexity

The recursive depth is

$$n$$

Therefore,

$$\boxed{O(n)}$$

Scalability Analysis

- Each new data element adds only **log n** additional work.
- However, the computation is repeated for every previous input.
- The total processing time becomes **n log n**.
- The algorithm scales efficiently for moderate and large datasets.
- It performs better than quadratic algorithms $O(n^2)$ but is slower than linear algorithms $O(n)$.

Asymptotic Interpretation

- **Big-O (Upper Bound):**

$$O(n \log n)$$

- **Big-Theta (Tight Bound):**

$$\Theta(n \log n)$$

- **Big-Omega (Lower Bound):**

$$\Omega(n \log n)$$

Thus,

$$T(n) = \Theta(n \log n)$$

Conclusion

Using the substitution method, the recurrence

$$T(n) = T(n - 1) + \log n$$

expands into

$$T(n) = 1 + \log(n!)$$

Applying Stirling's approximation,

$$T(n) = \Theta(n \log n)$$

The algorithm scales efficiently for big data analytics pipelines because each processing stage performs only logarithmic additional work, resulting in an overall time complexity of $\Theta(n \log n)$.

3. Screenshots to Include in the Report

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3
4 double T(int n)
5 {
6     if(n == 1)
7         return 1;
8
9     return T(n - 1) + log2(n);
10 }
11
12 int main()
13 {
14     int n;
15 }
```

Output

```
Enter the value of n: 8
T(8) = 16.30

=== Code Execution Successful ===
```

```
main.c
8
9     return T(n - 1) + log2(n);
10 }
11
12 int main()
13 {
14     int n;
15
16     printf("Enter the value of n: ");
17     scanf("%d", &n);
18
19     printf("T(%d) = %.2f\n", n, T(n));
20
21     return 0;
22 }
```

Output

```
Enter the value of n: 8
T(8) = 16.30

=== Code Execution Successful ===
```